

23 | 赋值兼容性的再说明

周爱民 (Aimingoo)

目录

- 1 条件表达式类型用于检查类型兼容的具体做法
- 2 兼容矩阵的输出
- 3 兼容性概念的再说明
- 4 总结

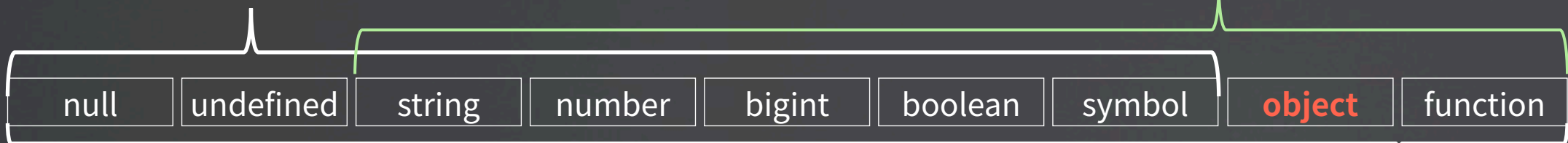
JS类型 => TS类型

Literal style

Construct style
(with/without package classes)

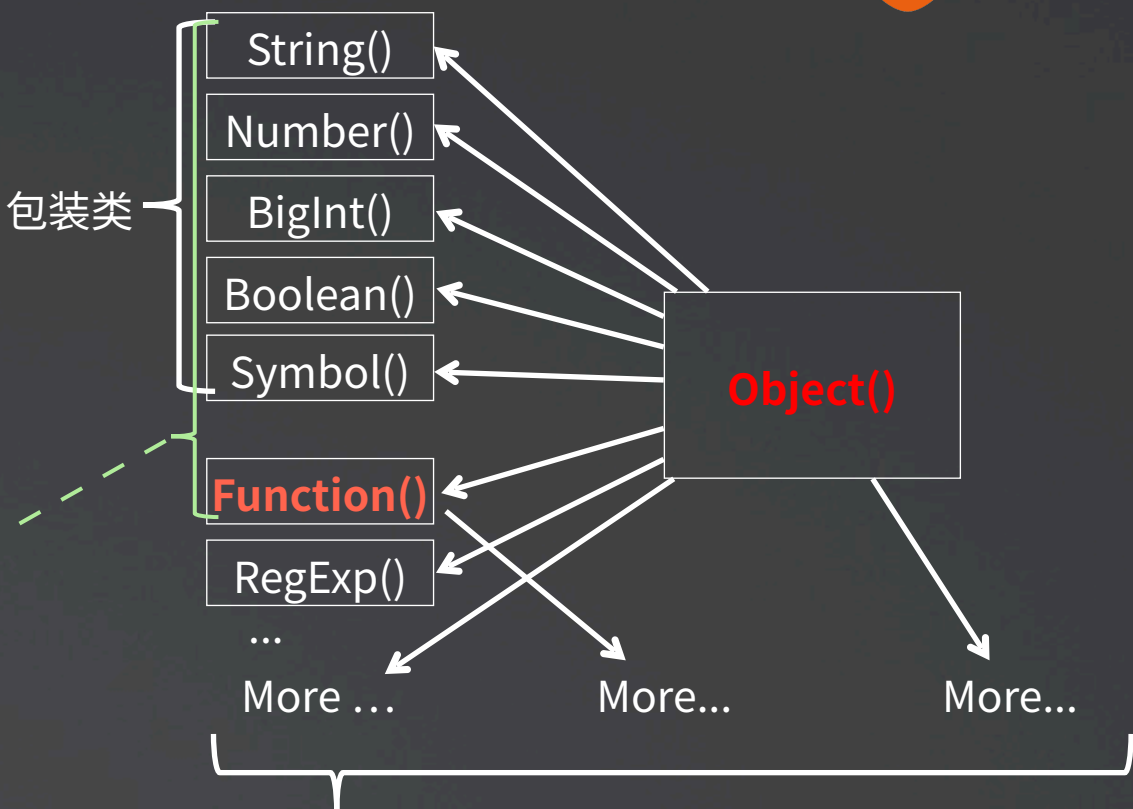


原始值类型(同es)
(Primitive types)



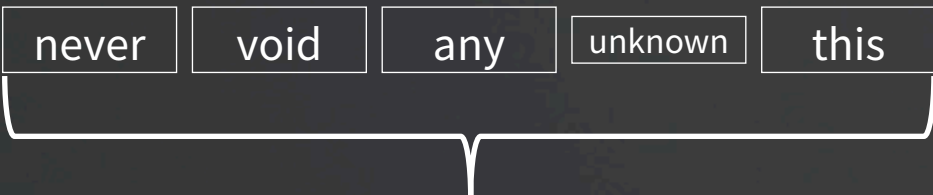
Above is collective type of
literal types

基础类型系统
(typeof)

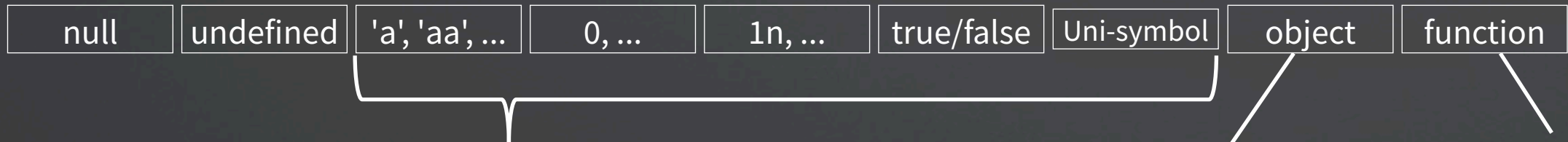


对象类型系统
(instanceof)

类/构造类型
(Class/Construct type)



特殊类型
(Special types)



字面类型
(Literal types)

字面对象: { ... }
数组与元组对象: [...]
字面正则对象: /.../...

对象类型
(object type)

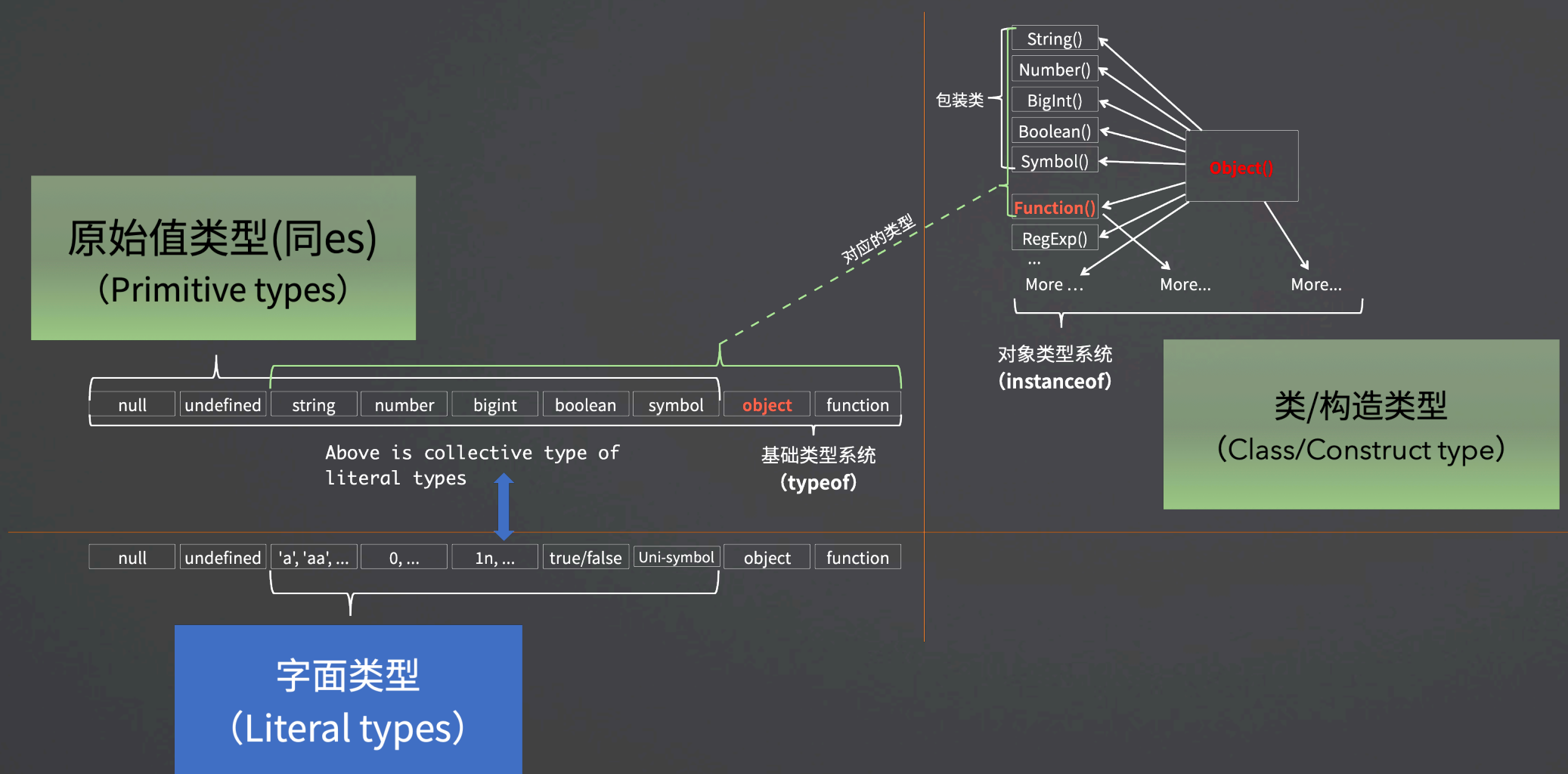
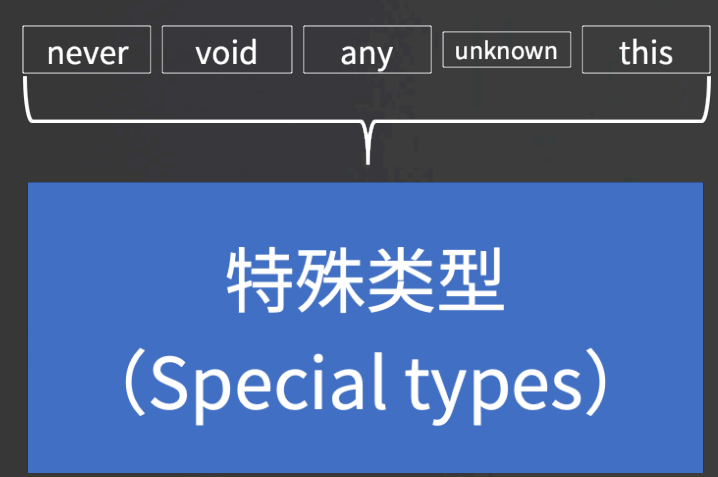
箭头函数: () => ...
具名函数: function f() { ... }
匿名函数: function () { ... }

函数类型
(function type)

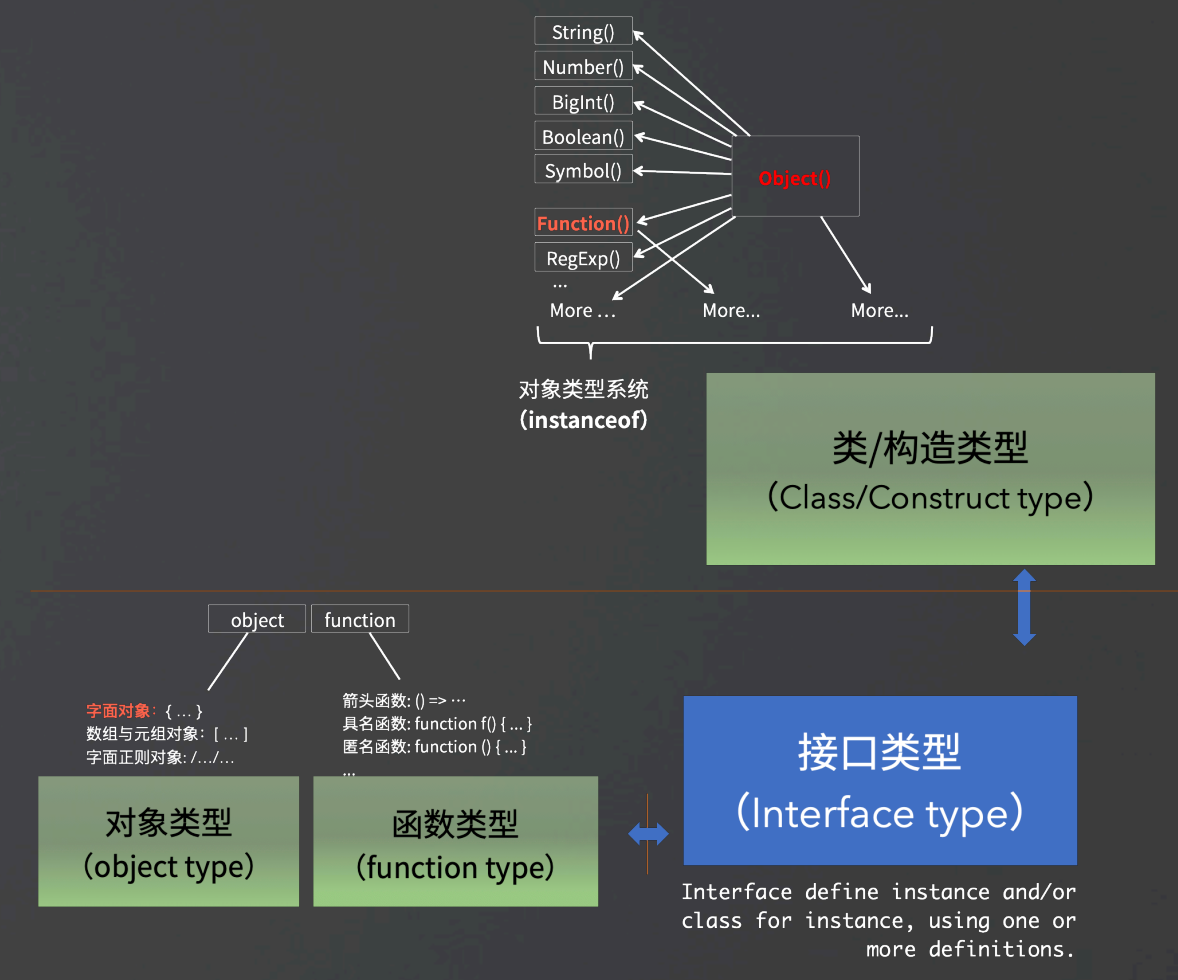
接口类型
(Interface type)

Interface define instance and/or
class for instance, using one or
more definitions.

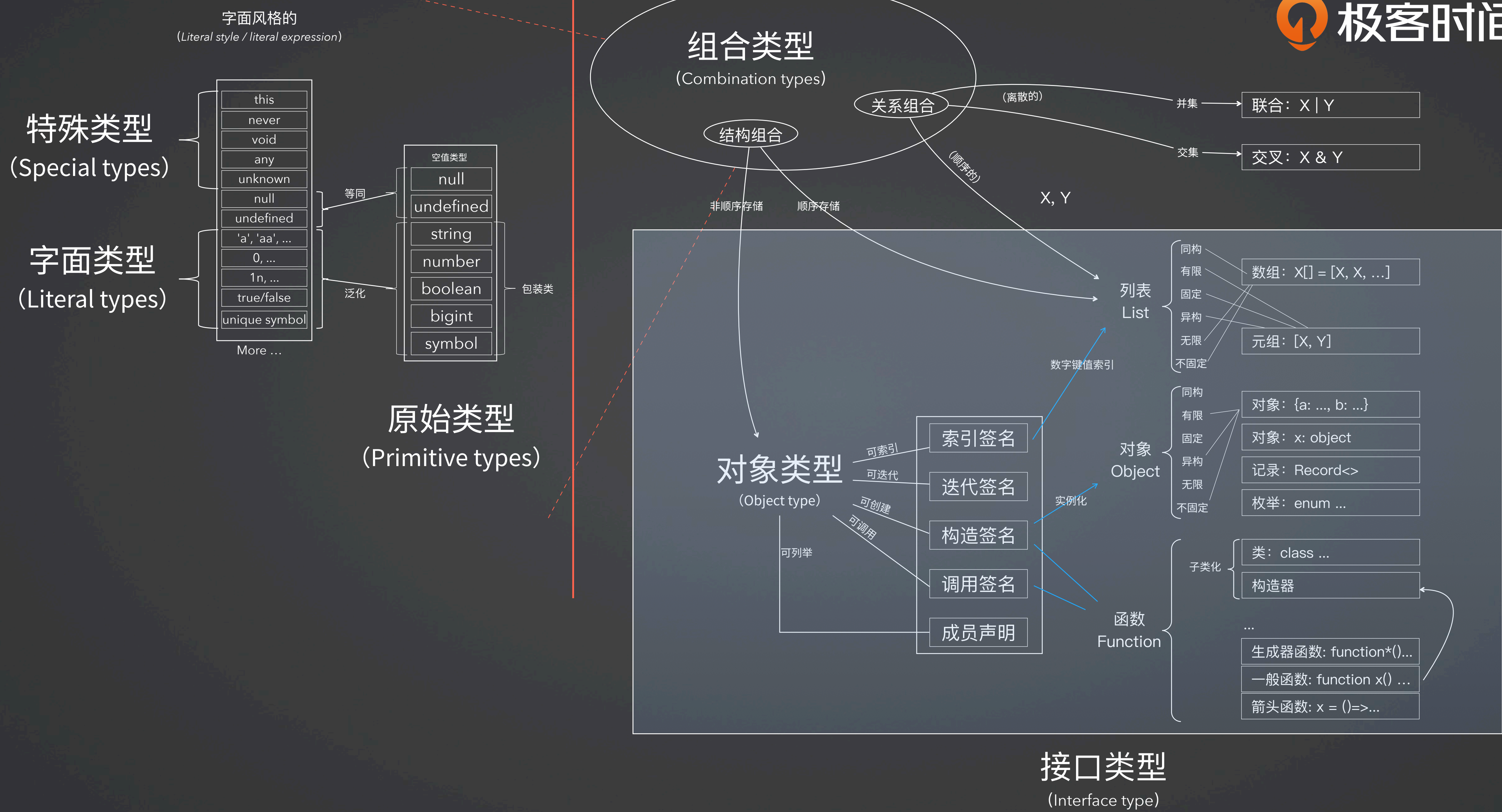
赋值兼容

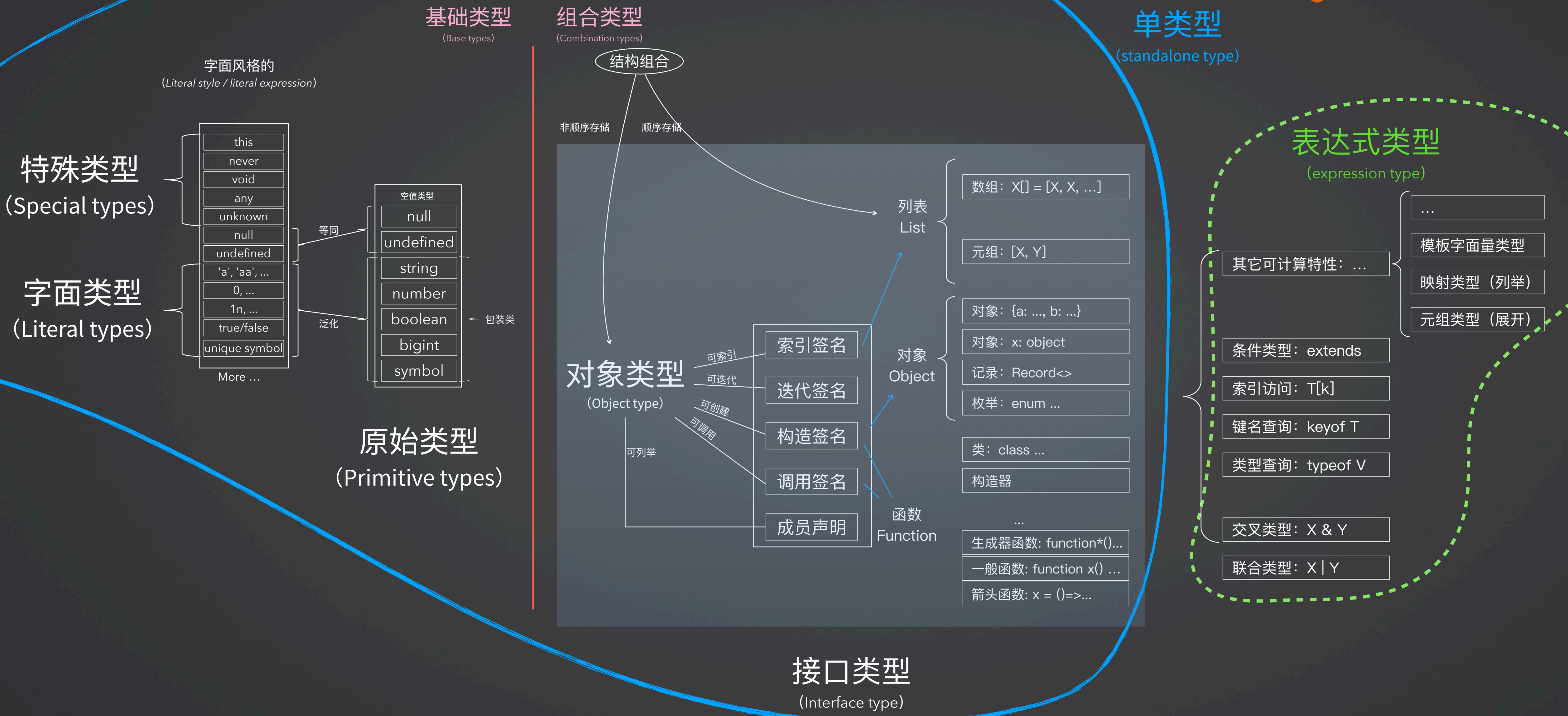


子类型兼容



结构类型兼容





简单类型间的赋值兼容性

x \ T	null	undefined	void	never	any	unknown	'a'	1	true	false	string
T_null	TRUE	FALSE	FALSE	TRUE	boolean	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
T_undefined	FALSE	TRUE	FALSE	TRUE	boolean	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
T_void	FALSE	TRUE	TRUE	TRUE	boolean	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
never	FALSE	FALSE	FALSE	TRUE	boolean	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
any	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
unknown	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
'a'	FALSE	FALSE	FALSE	TRUE	boolean	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE
1	FALSE	FALSE	FALSE	TRUE	boolean	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE
T_true	FALSE	FALSE	FALSE	TRUE	boolean	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE
T_false	FALSE	FALSE	FALSE	TRUE	boolean	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE
string	FALSE	FALSE	FALSE	TRUE	boolean	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE

- * never 可以赋给任何类型，但只能被 any 和 never 赋值
- * unknown 只能赋给 any 和 unknown，但可以被任何类型赋值

组合类型与赋值兼容性

{ ... } extends { ... } ? ...

()=>... extends (...)=> ... ? ...

signatures/list ... extends ... ? ...

表达式类型与赋值兼容性

$a|b|c$ extends $a\&b\&c$? ...

$a\&b\&c$ extends $a|b|c$? ...

$(A \text{ extends } B \text{ ? } \dots)$ extends

$(M \text{ extends } N \text{ ? } \dots)$? . : ..

总结

1. 赋值兼容通过“条件表达式”将结构兼容和子类型兼容统一在同一语义之下

- ▶ 语法 ``A extends B ? X : Y``
- ▶ 兼容性不再是静态的约定，它可以被计算了，并且这个计算过程可以用于驱动计算流程（分支和递归）

2. 注意TypeScript在处理“裸类型参数”时会有一些潜规则

- ▶ 元组的生成
- ▶ `keyof` 的处理
- ▶ `never` 和联合类型用在 `extends` 的左侧时的特殊性（上一讲）

作业

> 为 MapMatrix<> 返回的二维元组生成csv或xls文件，然后在表格软件中分析该矩阵。

> 为 MapMatrix<> 返回的二维元组添加首行和首列（标题行与标题列）。

// 提示1、向元组中添加新项的方法是使用展开运算 (...), 结果会是一个新元组

```
type X = [x, ...T]
```

// 提示2、新的泛型工具需要添加一个 类型约束，以使得元组在展开时通过类型检查。如下：

```
type MapMatrix2<Source extends any[]> = {  
  ...  
}
```

THANKS